



Department of Computer Science
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA

SUBJECT NAME:	COMPUTER ORGANIZATION AND ARCHITECTURE				
SUBJECT CODE:	SECR/SCSR 1033				
SEMESTER:	2 – 2025/2026				
LAB TITLE:	Lab 2: Arithmetic Equations & Operations				
STUDENT INFO :	Execute the lab in group of two. <table border="1"><thead><tr><th>Student 1</th><th>Student 2</th></tr></thead><tbody><tr><td>No. 1, 3, 5</td><td>No 2, 4, 6</td></tr></tbody></table> Name 1: <u>NASAAIE BIN NORISKAMAR</u> Metric No: <u>A25CS0118</u> Link for Video Demo: https://youtu.be/NdBCiJ206ew?si=xICRexI_LEBIH7YE Name 2: <u>MUHAMMAD FAIRUZ BIN HERMAN</u> Metric No: <u>A25CS0267</u> Link for Video Demo: https://youtu.be/xmFzMqeh1bc?si=6gjcvfQyliJh_6Ex	Student 1	Student 2	No. 1, 3, 5	No 2, 4, 6
Student 1	Student 2				
No. 1, 3, 5	No 2, 4, 6				
SUBMISSION DATE & ITEMS:	Duration of submission: - 2 weeks Submission items in e-learning: - 1. Lab 2 exercise sheet/file (in .pdf), with the links for demo video 5 - 10min, on the cover page. 3. The assembly programs (in .asm).				

MARKS:

Arithmetic Equation Coding in Assembly Language

- Q1. Execute the program below. Determine the program's output by inspecting the contents of the relevant registers.
- Fill in **Table 1** with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
 - Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
    mov ax, var1    ; LINE1
    mov bx, var2    ; LINE2
    xchg ax, bx     ; LINE3
    mov var1, ax    ; LINE4
    mov var2, bx    ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

Answer Q1

- a) Fill (Write) in the contents for the related register in each line:

Table 1

LINE1	AX = 001h var1 = 0001h	Move the value of var1 (1) into register AX
LINE2	BX = 0009h var2 = 0009h	move value of var2 to register BX
LINE3	AX = 0009h BX = 0001h	Swap the value of AX and BX using xchg
LINE4	AX = 0009h var1 = 0009h	Move new value of AX to variable var1
LINE5	BX = 0001h var2 = 0001h	move the new value of BX to the variable of var2

- b) Paste here a screenshot of all registers' content after each LINE is executed:

LINE1:
LINE2:
LINE3:
LINE4:
LINE5:

```
Registers
EAX = 001A0001 EBX = 00385000 ECX = 004010AA
EDX = 004010AA ESI = 004010AA EDI = 004010AA
EIP = 00403666 ESP = 001AFF78 EBP = 001AFF84
EFL = 00000246
```

```
Registers
EAX = 001A0001 EBX = 00380009 ECX = 004010AA
EDX = 004010AA ESI = 004010AA EDI = 004010AA
EIP = 0040366D ESP = 001AFF78 EBP = 001AFF84
EFL = 00000246
```

```
Registers
EAX = 001A0009 EBX = 00380001 ECX = 004010AA
EDX = 004010AA ESI = 004010AA EDI = 004010AA
EIP = 0040366F ESP = 001AFF78 EBP = 001AFF84
EFL = 00000246
```

Registers

EAX = 001A0009 EBX = 00380001 ECX = 004010AA
EDX = 004010AA ESI = 004010AA EDI = 004010AA
EIP = 00403675 ESP = 001AFF78 EBP = 001AFF84
EFL = 00000246

Registers

EAX = 001A0009 EBX = 00380001 ECX = 004010AA
EDX = 004010AA ESI = 004010AA EDI = 004010AA
EIP = 0040367C ESP = 001AFF78 EBP = 001AFF84
EFL = 00000246

- Q2. Execute the program below. Determine the program's output by inspecting the contents of the relevant registers and watches.
- Fill in **Table 2** with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
 - Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $Rval = (-Xval + (Yval - Zval)) + 1$

```
include irvine32.inc
```

```
.data
```

```
Rval DWORD ?
```

```
Xval DWORD 26
```

```
Yval DWORD 30
```

```
Zval DWORD 40
```

```
.code
```

```
main proc
```

```
    mov eax,Xval        ; LINE1
```

```
    neg eax             ; LINE2
```

```
    mov ebx,Yval        ; LINE3
```

```
    sub ebx,Zval        ; LINE4
```

```
    add eax,ebx         ; LINE5
```

```
    inc eax             ; LINE6
```

```
    mov Rval,eax        ; LINE7
```

```
    exit
```

```
main endp
```

```
end main
```

Answer Q2

- Fill (Write) in the contents for the related register in each line:

LINE1	EAX = 0000001Ah Xval = 0000001Ah	Move the value of Xval (26d) into register EAX
LINE2	EAX = FFFFFFFEh	Two's complement negation of EAX (-26d)
LINE3	EBX = 0000001Eh Yval = 0000001Eh	Move the value of Yval (30d) into register EBX
LINE4	EBX = FFFFFFF6h Zval = 00000028h	Subtract Zval (40d) from EBX (30-40 = -10)
LINE5	EAX = FFFFFFFDh EBX = FFFFFFF6h	Add EBX to EAX (-26 + (-10) = -36)
LINE6	EAX = FFFFFFF0h	Increment EAX by 1 (-36+1 = -35)
LINE7	EAX = FFFFFFF0h Rval = FFFFFFF0h	Move the final result in EAX to memory variable Rval

- Paste here screenshot of all registers' content after each LINE is executed:

LINE1: Registers

LINE2: EAX = 0000001A EBX = 002BE000 ECX = 004010AA

LINE3: EDX = 004010AA ESI = 004010AA EDI = 004010AA

LINE4: EIP = 00403665 ESP = 001AFF78 EBP = 001AFF84

LINE5: EFL = 00000246

LINE6: Registers

LINE7: EAX = FFFFFFFE EBX = 002BE000 ECX = 004010AA

EDX = 004010AA ESI = 004010AA EDI = 004010AA

EIP = 00403667 ESP = 001AFF78 EBP = 001AFF84

EFL = 00000293

0x00406008 = 0000001E

```
Registers
EAX = FFFFFFFE6 EBX = 0000001E ECX = 004010AA
EDX = 004010AA ESI = 004010AA EDI = 004010AA
EIP = 0040366D ESP = 001AFF78 EBP = 001AFF84
EFL = 00000293

0x0040600C = 00000028
```

```
Registers
EAX = FFFFFFFE6 EBX = FFFFFFFF6 ECX = 004010AA
EDX = 004010AA ESI = 004010AA EDI = 004010AA
EIP = 00403673 ESP = 001AFF78 EBP = 001AFF84
EFL = 00000287
```

```
Registers
EAX = FFFFFFFDC EBX = FFFFFFFF6 ECX = 004010AA
EDX = 004010AA ESI = 004010AA EDI = 004010AA
EIP = 00403675 ESP = 001AFF78 EBP = 001AFF84
EFL = 00000283
```

```
Registers
EAX = FFFFFFFDD EBX = FFFFFFFF6 ECX = 004010AA
EDX = 004010AA ESI = 004010AA EDI = 004010AA
EIP = 00403676 ESP = 001AFF78 EBP = 001AFF84
EFL = 00000287

0x00406000 = 00000000
```

```
Registers
EAX = FFFFFFFDD EBX = FFFFFFFF6 ECX = 004010AA
EDX = 004010AA ESI = 004010AA EDI = 004010AA
EIP = 0040367B ESP = 001AFF78 EBP = 001AFF84
EFL = 00000287
```

- Q3. Execute the program below. Determine the program's output by inspecting the contents of the relevant registers.
- Fill in **Table 3** with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
 - Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = [(\text{var1} * \text{var2}) + \text{var3}] - 1$

```
include Irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
    mov eax, var1        ; LINE1
    mul var2              ; LINE2
    add eax, var3         ; LINE3
    dec eax               ; LINE4
    exit
main endp
end main
```

Answer Q3

- Fill (Write) in the contents for the related register in each line:

Table 3

LINE1	EAX = 00000005h var1 = 00000005h	Move the value of var1 (5d) into register EAX
LINE2	EAX = 00000032h var2 = 0000000Ah	EAX time to the value of var2(5 x 10 = 50). 50d = 32h
LINE3	EAX = 00000046h var3 = 00000014h	Add var3(20d) to EAX (50 + 20 = 70). 70d = 46h
LINE4	EAX = 00000045h Var4 = 00000045h	Minus 1 from EAX (70 - 1 = 69). 69d = 45h

- Paste here screenshot of all registers' content after each LINE is executed:

LINE1:
LINE2:
LINE3:
LINE4:

```
Registers
EAX = 00000005 EBX = 002BB000 ECX = 004010AA
EDX = 004010AA ESI = 004010AA EDI = 004010AA
EIP = 00403665 ESP = 001AFF78 EBP = 001AFF84
EFL = 00000246

0x00406004 = 0000000A
```

```
Registers
EAX = 00000032 EBX = 002BB000 ECX = 004010AA
EDX = 00000000 ESI = 004010AA EDI = 004010AA
EIP = 0040366B ESP = 001AFF78 EBP = 001AFF84
EFL = 00000202

0x00406008 = 00000014
```

Registers ▼ 🔍

EAX = 00000046 EBX = 002BB000 ECX = 004010AA
EDX = 00000000 ESI = 004010AA EDI = 004010AA
EIP = 00403671 ESP = 001AFF78 EBP = 001AFF84
EFL = 00000202

Registers ▼ 🔍

EAX = 00000045 EBX = 002BB000 ECX = 004010AA
EDX = 00000000 ESI = 004010AA EDI = 004010AA
EIP = 00403672 ESP = 001AFF78 EBP = 001AFF84
EFL = 00000202

0x0040600C = 00000000

- Q4. Execute the program below. Determine the program's output by inspecting the contents of the relevant registers.
- Fill in **Table 4** with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.
 - Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$

```
include irvine32.inc
.data
    var1 WORD 40
    var2 WORD 10
    var4 WORD ?
.code
main proc
    mov ax,var1      ; LINE1
    mov bx,5         ; LINE2
    mul bx           ; LINE3
    mov bx,var2      ; LINE4
    sub bx,3         ; LINE5
    div bx           ; LINE6
    mov var4,ax      ; LINE7
    exit
main endp
end main
```

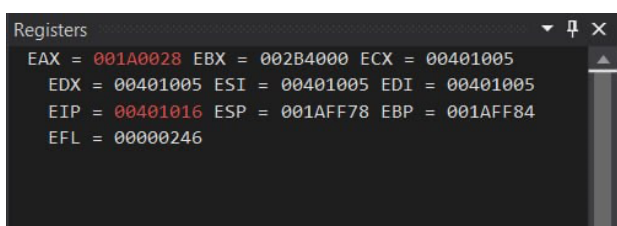
Answer Q4

- Fill (Write) in the contents for the related register in each line:

LINE1	AX = 0028h var1 = 0028h	Move the value of var1 (40d) into register AX
LINE2	BX = 5h	Move the value of 5 into register BX
LINE3	AX = 0028h BX = 08h	Multiply the value of AX with BX and store into register BX
LINE4	BX = 000Ah var2 = 000Ah	Move the value of var2 (10d) into register AX
LINE5	BX = 0007h	Subtract the value of 3 with BX (10d) and store into register BX
LINE6	AX = 0005h BX = 0007h DX = 0005h	Divide BX with AX and store the quotient result to AX while remainder store into DX.
LINE7	AX = 0000h var4 = 0000h	Move the value of var4(?) into register AX.

- Paste here screenshot of all registers' content after each LINE is executed:

LINE1:
LINE2:
LINE3:



Registers ▾ ⓘ

EAX = 001A0028 EBX = 002B0005 ECX = 00401005
EDX = 00401005 ESI = 00401005 EDI = 00401005
EIP = 0040101A ESP = 001AFF78 EBP = 001AFF84
EFL = 00000246

Registers ▾ ⓘ

EAX = 001A00C8 EBX = 002B0005 ECX = 00401005
EDX = 00400000 ESI = 00401005 EDI = 00401005
EIP = 0040101D ESP = 001AFF78 EBP = 001AFF84
EFL = 00000202

Registers ▾ ⓘ

EAX = 001A00C8 EBX = 002B000A ECX = 00401005
EDX = 00400000 ESI = 00401005 EDI = 00401005
EIP = 00401024 ESP = 001AFF78 EBP = 001AFF84
EFL = 00000202

Registers ▾ ⓘ

EAX = 001A00C8 EBX = 002B0007 ECX = 00401005
EDX = 00400000 ESI = 00401005 EDI = 00401005
EIP = 00401028 ESP = 001AFF78 EBP = 001AFF84
EFL = 00000202

Registers ▾ ⓘ

EAX = 001A001C EBX = 002B0007 ECX = 00401005
EDX = 00400004 ESI = 00401005 EDI = 00401005
EIP = 0040102B ESP = 001AFF78 EBP = 001AFF84
EFL = 00000202

Registers ▾ ⓘ

EAX = 001A001C EBX = 002B0007 ECX = 00401005
EDX = 00400004 ESI = 00401005 EDI = 00401005
EIP = 00401031 ESP = 001AFF78 EBP = 001AFF84
EFL = 00000202

LINE4:
LINE5:
LINE6:
LINE7:

Short Notes for MUL CX and DIV BL:

MUL CX

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
- b. The operand size determines the size of the result:
 - i. Byte-sized operand: Result in AX
 - ii. Word-sized operand: Result in DX:AX
 - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
 - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
- b. The divisor is specified as the operand of the DIV instruction.
- c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
- d. The remainder is stored in DX.
- e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
- f. If the divisor is 0, a division error occurs.
- g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.

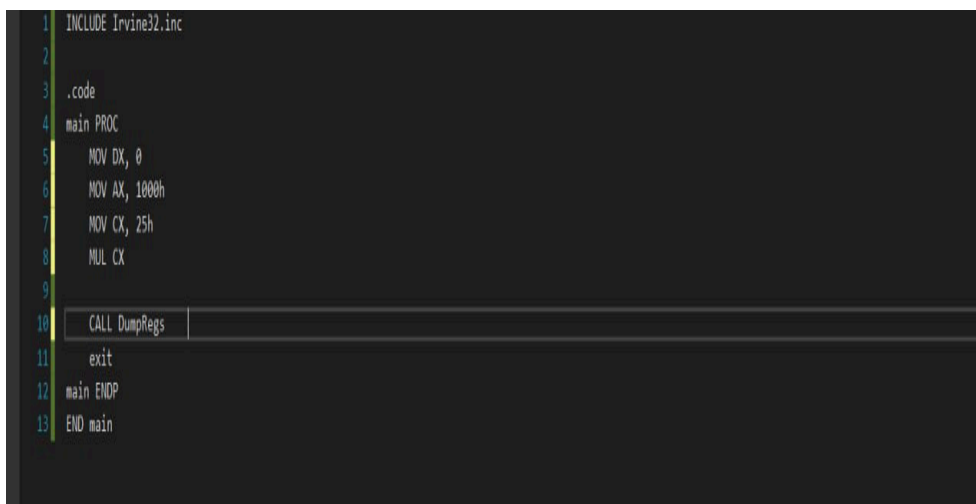
- Q5. Given the following instructions as is, Code Snippet 1.
- Write a full program to execute Code Snippet 1.
 - What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

; Code Snippet 1 (MUL CX)

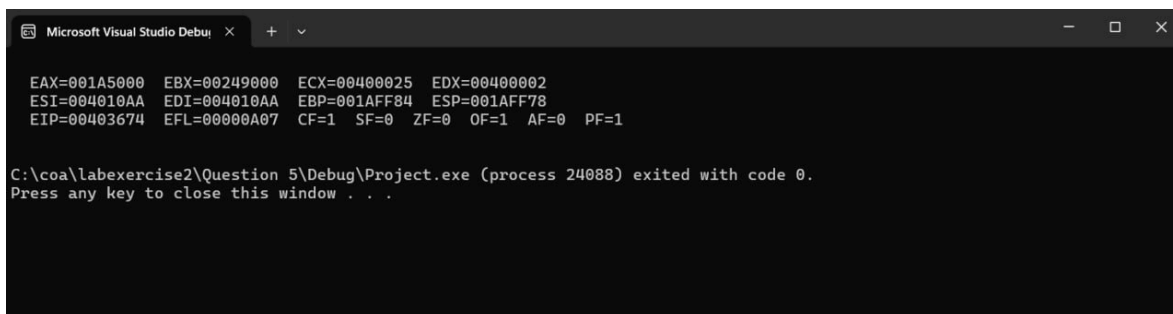
```
MOV DX, 0           ; Clear DX
MOV AX, 1000h        ; Load 1000h into AX
MOV CX, 25h          ; Load 25h into CX
MUL CX               ; Multiply AX by CX, storing the result in DX:AX
```

Answer Q5

- Screenshot of full program (.asm) :
- Paste here the screenshot of the final registers' content (DumpReg):



```
1  INCLUDE Irvine32.inc
2
3  .code
4  main PROC
5      MOV DX, 0
6      MOV AX, 1000h
7      MOV CX, 25h
8      MUL CX
9
10     CALL DumpRegs
11     exit
12 main ENDP
13 END main
```



Microsoft Visual Studio Debug Console

```
EAX=001A5000  EBX=00249000  ECX=00400025  EDX=00400002
ESI=004010AA  EDI=004010AA  EBP=001AFF84  ESP=001AFF78
EIP=00403674  EFL=00000A07  CF=1  SF=0  ZF=0  OF=1  AF=0  PF=1
```

C:\coa\labexercise2\Question 5\Debug\Project.exe (process 24088) exited with code 0.
Press any key to close this window . . .

- Q6. Given the following instructions as is, Code Snippet 2.
- Write a full program to execute Code Snippet 2.
 - What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

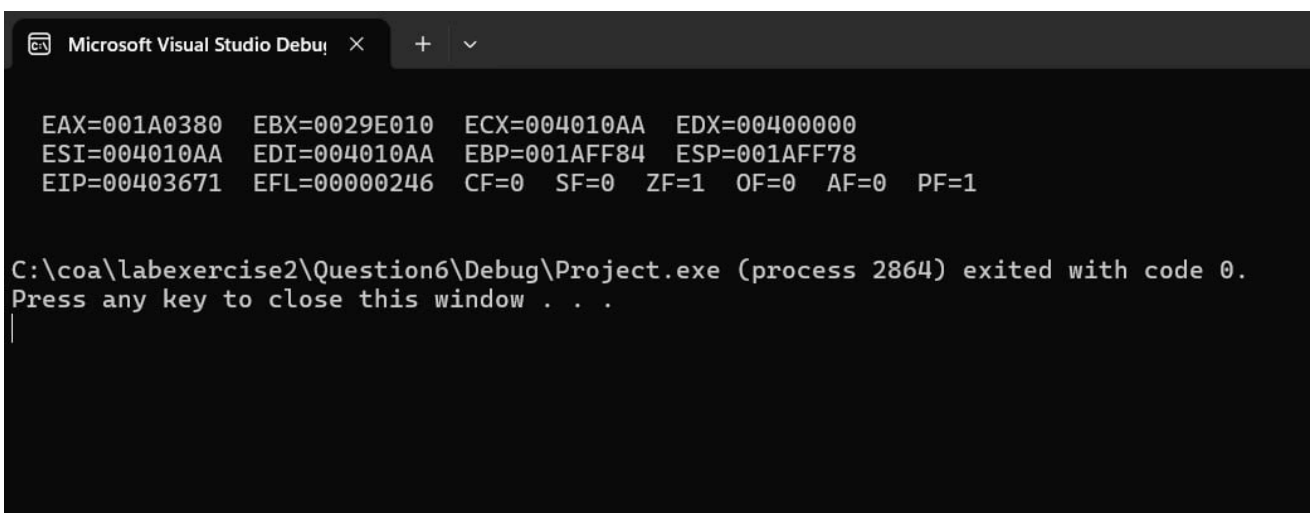
; Code Snippet 2 (DIV BL)

```
MOV DX, 0      ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h   ; Load the dividend (8003h) into AX
MOV BL, 10h    ; Load the divisor (10h) into BL
DIV BL         ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

Answer Q6

- Screenshot of full program (.asm) :
- Paste here the screenshot of the final registers' content (DumpReg):

```
1  include irvine32.inc
2  .code
3  main proc
4  MOV DX, 0 ; Clear DX to form the 16-bit dividend in DX:AX
5  MOV AX, 803h ; Load the dividend (8003h) into AX
6  MOV BL, 10h ; Load the divisor (10h) into BL
7  DIV BL ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
8  exit
9  main endp
10 end main
```



The screenshot shows the Microsoft Visual Studio Debugger window. At the top, the title bar reads "Microsoft Visual Studio Debug". Below the title bar, the register values are displayed in a grid:

EAX=001A0380	EBX=0029E010	ECX=004010AA	EDX=00400000
ESI=004010AA	EDI=004010AA	EBP=001AFF84	ESP=001AFF78
EIP=00403671	EFL=00000246	CF=0	SF=0 ZF=1 OF=0 AF=0 PF=1

Below the register values, the following text is displayed:

```
C:\coa\labexercise2\Question6\Debug\Project.exe (process 2864) exited with code 0.
Press any key to close this window . . .
```